TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN
MÔN: KIỂM THỬ PHẦN MỀM

Assignment 1: Kiểm Thử Hộp Đen
Thiết Kế Test Case & Tự Động Hóa Web Với JUnit 5

Hệ thống kiểm thử mục tiêu: SauceDemo (https://www.saucedemo.com)

Nhóm thực hiện: Nhóm 04

Giảng viên hướng dẫn: Từ Lãng Phiêu

STT	Họ và tên sinh viên	Vai trò phụ trách	MSSV	Đóng góp
1	Lê Khánh Đăng	Nhóm trưởng (Login - FR-01)	3124560022	100%
2	Dương Chí Tài	QA (Giỏ hàng - FR-02)	3124560076	100%
3	Trần Việt Đức	QA (Thanh toán - FR-03)	3124560024	100%
4	Nguyễn Thành Nhân	QA (Sắp xếp - FR-04)	3124560065	100%
5	Phan Ngọc Vinh	QA (Hiệu năng - NFR-01/Excel)	3124560098	100%

TP. HỒ CHÍ MINH, THÁNG 10/2026

Mục lục

1 Tổng quan hệ thống và môi trường kiểm thử **2**
1.1 Danh sách tài khoản kiểm thử của hệ thống . . . 2
1.2 Môi trường thực thi kiểm thử của nhóm . . . 2

2 NV1 – Chạy và mở rộng mã nguồn mẫu **2**
2.1 Kết quả thực thi mã nguồn mẫu ban đầu . . . 2
2.2 Vai trò của các Annotation trong JUnit 5 . . . 3

3 NV2 – Phân tích yêu cầu và Đánh giá rủi ro **4**
3.1 Đặc tả yêu cầu chi tiết (Requirements Specification) . . . 4
3.1.1 Đặc tả FR-01: Xác thực và Đăng nhập (Lê Khánh Đăng) . . . 4
3.1.2 Đặc tả FR-02: Quản lý giỏ hàng (Dương Chí Tài) . . . 4
3.1.3 Đặc tả FR-03: Quy trình thanh toán (Trần Việt Đức) . . . 5
3.1.4 Đặc tả FR-04: Sắp xếp danh sách sản phẩm (Nguyễn Thành Nhân) . . . 5
3.1.5 Đặc tả NFR-01: Yêu cầu phi chức năng - Hiệu năng (Phan Ngọc Vinh) . . . 6
3.2 Đánh giá rủi ro (Risk-based Testing) . . . 7
3.3 Ma trận truy vết yêu cầu (Traceability Matrix) . . . 7

4 NV3 – Thiết kế Test Case **8**
4.1 Phân lớp tương đương (Equivalence Partitioning - EP) . . . 8
4.1.1 Phân lớp tương đương cho trường Username (FR-01) . . . 8
4.1.2 Phân lớp tương đương tự mở rộng cho trường Password (FR-01) . . . 8
4.1.3 Phân lớp tương đương cho chức năng Sắp xếp (FR-04) . . . 8
4.2 Phân tích giá trị biên (Boundary Value Analysis - BVA) . . . 9
4.2.1 Miền 1: Số lượng sản phẩm trong giỏ hàng (FR-02) . . . 9
4.2.2 Miền 2: Làm tròn thuế 8% tại trang Thanh toán (FR-03) . . . 9
4.3 Bảng quyết định (Decision Table - DT) . . . 10
4.3.1 Bảng quyết định 1: Logic xác thực Đăng nhập (FR-01) . . . 10
4.3.2 Bảng quyết định 2: Xác thực biểu mẫu thông tin Thanh toán (FR-03) . . . 10
4.4 Kiểm thử ca sử dụng (Use Case Testing) . . . 11
4.4.1 Ca sử dụng 1 (UC1): Quy trình thanh toán đơn hàng (FR-03) . . . 11
4.4.2 Ca sử dụng 2 (UC2): Quản lý giỏ hàng (FR-02) [Nhóm tự mở rộng] . . . 13

5 NV4 – Thực thi thủ công và Báo cáo lỗi **14**
5.1 Nhật ký thực thi kiểm thử thủ công . . . 14
5.2 Báo cáo các lỗi hệ thống phát hiện được (Defect Reports) . . . 15

6 NV5 – Tự động hóa bằng JUnit 5 **18**
6.1 Kiến trúc Page Object Model (POM) . . . 18
6.2 Trích đoạn mã nguồn các lớp kiểm thử tiêu biểu . . . 18
6.2.1 Lớp LoginTest.java (Sử dụng CsvFileSource và CsvSource) . . . 18
6.2.2 Lớp CheckoutTest.java (Kiểm thử kịch bản Use Case) . . . 19
6.2.3 Lớp CartTest.java (Kiểm thử giá trị biên BVA giỏ hàng - FR-02) . . . 20
6.2.4 Lớp SortTest.java (Kiểm thử sắp xếp và phát hiện lỗi Sort - FR-04) . . . 21
6.2.5 Lớp PerformanceTest.java (Đo hiệu năng NFR-01 & Đánh dấu lỗi đã biết) . . . 22
6.3 Kết quả thực thi tự động (Maven Surefire Report) . . . 23
6.4 Cấu hình mở rộng đạt điểm Bonus (Mục 7.7 đề bài) . . . 23
6.4.1 Cơ chế tự động chụp ảnh màn hình khi FAIL bằng JUnit 5 TestWatcher . . . 23
6.4.2 Tích hợp quy trình kiểm thử tự động liên tục (CI) với GitHub Actions . . . 24

7 Kết luận và Tự đánh giá trước khi nộp bài **25**

1 Tổng quan hệ thống và môi trường kiểm thử

Hệ thống được chọn để thực hiện kiểm thử trong đề tài là **SauceDemo** (https://www.saucedemo.com) — trang web bán hàng mẫu chuẩn công nghiệp chuyên dùng để giảng dạy và thực hành kiểm thử phần mềm tự động.

1.1 Danh sách tài khoản kiểm thử của hệ thống

Trên hệ thống SauceDemo, hệ thống cung cấp sẵn 6 tài khoản mẫu với mật khẩu chung `secret_sauce`. Mỗi tài khoản được thiết kế để mô phỏng một trạng thái hoặc lỗi đặc thù trong quá trình kiểm thử:

Tài khoản	**Hành vi và mục đích dùng để kiểm thử**
`standard_user`	Tài khoản tiêu chuẩn, hoạt động bình thường trên mọi chức năng; dùng cho luồng chính (Happy Path) từ đăng nhập, giỏ hàng đến thanh toán.
`locked_out_user`	Tài khoản bị khóa; đăng nhập đúng mật khẩu vẫn bị từ chối; dùng để kiểm thử nhánh lỗi tài khoản bị khóa trong FR-01.
`problem_user`	Tài khoản có lỗi giao diện (ảnh sản phẩm bị lỗi hiển thị giống nhau) và lỗi chức năng (không sắp xếp được sản phẩm, không xóa được sản phẩm); dùng để săn lỗi ở NV4.
`performance_glitch_user`	Bị làm chậm có chủ đích ($\approx$ 5 giây); dùng để kiểm thử yêu cầu phi chức năng hiệu năng NFR-01 (bắt lỗi **BUG-01**).
`error_user`	Tài khoản gặp lỗi ngoại lệ khi thao tác (ô Last Name không nhận ký tự nhưng vẫn cho phép Continue); dùng cho kiểm thử khám phá mở rộng.
`visual_user`	Tài khoản có lỗi hiển thị sai lệch vị trí CSS và giá sản phẩm bị gán ngẫu nhiên; dùng cho kiểm thử hồi quy trực quan.

Bảng 1: Danh sách đầy đủ 6 tài khoản kiểm thử trên SauceDemo

Ghi chú phạm vi: Theo đặc tả tại Mục 2.1 của đề bài, nhóm tập trung trọng tâm vào **4 tài khoản đầu tiên** (`standard_user`, `locked_out_user`, `problem_user`, và `performance_glitch_user`) để hoàn thành các yêu cầu từ FR-01 đến NFR-01. Hai tài khoản còn lại được nhóm sử dụng để mở rộng kiểm thử khám phá và phát hiện lỗi hệ thống.

1.2 Môi trường thực thi kiểm thử của nhóm

- **Hệ điều hành:** Microsoft Windows 11 Pro 64-bit.
- **Bộ phát triển Java (JDK):** OpenJDK Temurin 21.0.12 LTS.
- **Công cụ quản lý dự án:** Apache Maven 3.9+.
- **Trình duyệt kiểm thử:** Google Chrome Version 155.0.8059.40 (64-bit) và Mozilla Firefox.
- **Thư viện kiểm thử cốt lõi:** JUnit 5.11 (Jupiter) và Selenium WebDriver 4.27.

2 NV1 – Chạy và mở rộng mã nguồn mẫu

2.1 Kết quả thực thi mã nguồn mẫu ban đầu

Nhóm tiến hành khởi chạy dự án mẫu (`starter-code`) thông qua giao diện dòng lệnh bằng lệnh `mvn clean test`. Hệ thống tích hợp Selenium Manager tự động thiết lập driver phù hợp và thực thi thành công các ca kiểm thử mẫu ở chế độ ngầm (*headless*).

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 9.074 s -- in vn.edu.softwaretesting.tests.LoginTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  13.372 s
[INFO] Finished at: 2026-10-12T21:14:55+07:00
[INFO] ------------------------------------------------------------------------
```

Hình 1: Ảnh chụp màn hình kết quả chạy thực tế `mvn clean test` thành công

Sau khi mở rộng tệp `login-data.csv` với các trường hợp biên (chuỗi rỗng), toàn bộ các ca kiểm thử đều vượt qua thành công (**BUILD SUCCESS**).

2.2 Vai trò của các Annotation trong JUnit 5

Kiến trúc kiểm thử tự động của dự án ứng dụng ba annotation cốt lõi của JUnit 5:

a) `@BeforeEach` (Gói `org.junit.jupiter.api`):
Được thực thi tự động ngay trước mỗi phương thức kiểm thử. Trong lớp cơ sở `WebTestBase`, annotation này đảm nhận việc khởi tạo phiên bản trình duyệt `WebDriver` mới tinh, cấu hình kích thước cửa sổ và timeout. Nhờ vậy, mỗi ca kiểm thử được chạy độc lập hoàn toàn (*Test Isolation*), không bị ảnh hưởng bởi cookie, cache hay dữ liệu phiên của ca kiểm thử chạy trước đó.

b) `@ParameterizedTest` (Gói `org.junit.jupiter.params`):
Cho phép thực thi lặp lại cùng một phương thức kiểm thử với nhiều bộ tham số đầu vào khác nhau. Kỹ thuật này áp dụng triệt để nguyên lý DRY (*Don't Repeat Yourself*), tránh việc sao chép mã nguồn kiểm thử và hỗ trợ định dạng tên ca kiểm thử linh hoạt qua thuộc tính `name`.

c) `@CsvFileSource` (Gói `org.junit.jupiter.params.provider`):
Đóng vai trò là bộ nạp tham số đọc dữ liệu từ tệp tin dạng bảng CSV ngoài mã nguồn (`login-data.csv`). Cơ chế này giúp tách bạch hoàn toàn giữa **Mã kịch bản kiểm thử (Test Script)** và **Dữ liệu kiểm thử (Test Data)**, cho phép bổ sung hàng loạt ca kiểm thử mới trực tiếp trên tệp CSV mà không cần biên dịch lại mã nguồn Java.

3 NV2 – Phân tích yêu cầu và Đánh giá rủi ro

3.1 Đặc tả yêu cầu chi tiết (Requirements Specification)

Nhóm phân rã đặc tả hệ thống thành 5 phân hệ tương ứng với các thành viên phụ trách, mỗi phân hệ được chuẩn hóa theo định dạng 3 cột thống nhất: **Mã yêu cầu**, **Mô tả yêu cầu**, và **Kết quả mong đợi (Quy tắc kiểm tra cụ thể)**.

3.1.1 Đặc tả FR-01: Xác thực và Đăng nhập (Lê Khánh Đăng)

Mã	Mô tả yêu cầu	Kết quả mong đợi (Quy tắc kiểm tra cụ thể)
FR-01.1	Đăng nhập thành công với tài khoản hợp lệ	Nhập `standard_user` / `secret_sauce` → Chuyển hướng sang URL `https://www.saucedemo.com/inventory.html`, hiển thị đầy đủ 6 sản phẩm và menu điều hướng.
FR-01.2	Kiểm tra để trống trường Username	Bỏ trống username → Giữ nguyên trang đăng nhập; hiển thị thông báo lỗi: *"Epic sadface: Username is required"*.
FR-01.3	Kiểm tra để trống trường Password	Có username nhưng để trống password → Giữ nguyên trang đăng nhập; hiển thị thông báo lỗi: *"Epic sadface: Password is required"*.
FR-01.4	Kiểm tra thông tin sai hoặc sai chữ hoa/thường	Nhập sai tên, sai mật khẩu, hoặc sai chữ hoa/thường → Hiển thị lỗi: *"Epic sadface: Username and password do not match any user in this service"*.
FR-01.5	Đăng nhập tài khoản bị khóa	Nhập `locked_out_user` / `secret_sauce` → Hiển thị lỗi: *"Epic sadface: Sorry, this user has been locked out."*.

Bảng 2: Bảng đặc tả yêu cầu chức năng FR-01: Đăng nhập

3.1.2 Đặc tả FR-02: Quản lý giỏ hàng (Dương Chí Tài)

Mã	Mô tả yêu cầu	Kết quả mong đợi (Quy tắc kiểm tra cụ thể)
FR-02.1	Trạng thái nút Add to cart	Sản phẩm chưa có trong giỏ sẽ hiển thị nút "Add to cart". Khi nhấn nút, sản phẩm được thêm vào giỏ; nút tại trang `inventory.html` chuyển thành "Remove" và badge giỏ hàng tăng thêm 1.
FR-02.2	Trạng thái nút Remove	Khi người dùng nhấn nút "Remove" (tại `inventory.html` hoặc `cart.html`), sản phẩm tương ứng bị xóa khỏi giỏ; nút đổi trở lại "Add to cart" và badge giỏ hàng giảm đi 1.
FR-02.3	Badge hiển thị số lượng từ 1 đến 6	Khi giỏ có từ 1 đến 6 sản phẩm, badge hiển thị đúng số lượng sản phẩm hiện có (ví dụ: có 3 sản phẩm thì badge hiển thị số 3; có đủ 6 sản phẩm thì badge hiển thị số 6).
FR-02.4	Giỏ hàng rỗng	Khi chưa có sản phẩm nào được thêm vào giỏ hoặc khi tất cả sản phẩm đã bị xóa, badge không hiển thị (ẩn hoàn toàn khỏi DOM). Nút của các sản phẩm hiển thị lại "Add to cart".

Bảng 3: Bảng đặc tả yêu cầu chức năng FR-02: Giỏ hàng

3.1.3 Đặc tả FR-03: Quy trình thanh toán (Trần Việt Đức)

Mã	Mô tả yêu cầu	Kết quả mong đợi (Quy tắc kiểm tra cụ thể)
FR-03.1	Kiểm tra trường First Name	First Name là bắt buộc; khi để trống cả 3 trường hoặc để trống First Name, hệ thống ưu tiên hiển thị lỗi: *"Error: First Name is required"* và giữ nguyên trang `checkout-step-one.html`.
FR-03.2	Kiểm tra trường Last Name	Nếu First Name hợp lệ nhưng Last Name để trống, hệ thống hiển thị lỗi: *"Error: Last Name is required"* và giữ nguyên trang `checkout-step-one.html`.
FR-03.3	Kiểm tra trường Postal Code	Nếu cả First Name và Last Name đều hợp lệ nhưng Postal Code để trống, hệ thống hiển thị lỗi: *"Error: Postal Code is required"* và giữ nguyên trang `checkout-step-one.html`.
FR-03.4	Tính tiền thuế (Tax)	Thuế được tính bằng 8% Item Total và làm tròn chuẩn 2 chữ số thập phân: Tax = round(Item Total $\times$ 0.08, 2).
FR-03.5	Tính tổng thanh toán (Total)	Tổng tiền hiển thị bằng tổng tiền hàng cộng tiền thuế: Total = Item Total + Tax.
FR-03.6	Hoàn tất đơn hàng (Finish)	Chuyển đến URL `checkout-complete.html`; hiển thị thông báo *"THANK YOU FOR YOUR ORDER!"*; giỏ hàng được làm rỗng và badge giỏ hàng biến mất.
FR-03.7	Hủy tại bước nhập thông tin (Cancel)	Nhấn Cancel tại `checkout-step-one.html` sẽ chuyển hướng về URL `cart.html`; các sản phẩm đã chọn vẫn còn nguyên vẹn trong giỏ hàng.

Bảng 4: Bảng đặc tả yêu cầu chức năng FR-03: Thanh toán

3.1.4 Đặc tả FR-04: Sắp xếp danh sách sản phẩm (Nguyễn Thành Nhân)

Mã	Mô tả yêu cầu	Kết quả mong đợi (Quy tắc kiểm tra cụ thể)
FR-04.1	Sắp xếp theo Name (A to Z) [Mặc định]	Sắp xếp sản phẩm theo tên tăng dần A–Z (mặc định khi mới vào trang). Với 3 sản phẩm mẫu: Sauce Labs Backpack $\rightarrow$ Sauce Labs Bike Light $\rightarrow$ Sauce Labs Bolt T-Shirt.
FR-04.2	Sắp xếp theo Name (Z to A)	Sắp xếp sản phẩm theo tên giảm dần Z–A. Danh sách hiển thị đảo ngược: Sauce Labs Bolt T-Shirt $\rightarrow$ Sauce Labs Bike Light $\rightarrow$ Sauce Labs Backpack.
FR-04.3	Sắp xếp theo Price (low to high)	Sắp xếp sản phẩm theo giá từ thấp tới cao. Sản phẩm đầu danh sách có giá thấp nhất là \$7.99 (Sauce Labs Onesie), sản phẩm cuối cùng là \$49.99 (Sauce Labs Fleece Jacket). Giá sản phẩm phía trên luôn $\leq$ sản phẩm phía dưới.
FR-04.4	Sắp xếp theo Price (high to low)	Sắp xếp sản phẩm theo giá từ cao tới thấp. Sản phẩm đầu danh sách có giá cao nhất là \$49.99 (Sauce Labs Fleece Jacket), sản phẩm cuối cùng là \$7.99 (Sauce Labs Onesie). Giá sản phẩm phía trên luôn $\geq$ sản phẩm phía dưới.

Bảng 5: Bảng đặc tả yêu cầu chức năng FR-04: Sắp xếp sản phẩm

3.1.5 Đặc tả NFR-01: Yêu cầu phi chức năng - Hiệu năng (Phan Ngọc Vinh)

Mã	Mô tả yêu cầu	Kết quả mong đợi (Quy tắc kiểm tra cụ thể)
NFR-01.1	Tiêu chuẩn thời gian hiển thị trang sản phẩm	Sau khi người dùng nhấn Login với thông tin đúng, trang sản phẩm (`inventory.html`) phải hiển thị hoàn tất trong thời gian không vượt quá 3 giây. Tiêu chí chấp nhận: Trung vị thời gian đo $\leq$ 3.000 ms (3.0s).
NFR-01.2	Điều kiện và phương pháp đo kiểm	Đăng nhập tài khoản `performance_glitch_user` / `secret_sauce` trên mạng cáp quang ổn định. Thực hiện đo độc lập 5 lần liên tiếp từ thời điểm nhấn nút Login đến khi DOM trang tải xong, lấy giá trị trung vị của 5 lần đo.

Bảng 6: Bảng đặc tả yêu cầu phi chức năng NFR-01: Hiệu năng

3.2 Đánh giá rủi ro (Risk-based Testing)

Mức độ rủi ro R được xác định theo công thức chuẩn:

$$R = P \times I \tag{1}$$

Trong đó P là xác suất xảy ra lỗi (1–5) và I là mức độ tác động thiệt hại (1–5).

Mã	Chức năng	P	I	R	Mức độ	Lý do đánh giá
FR-03	Thanh toán	3	5	**15**	Rất cao	Tính tiền/thuế sai gây thất thoát tài chính trực tiếp; luồng nhập liệu có nhiều bước phức tạp.
FR-01	Đăng nhập	3	4	**12**	Cao	Cửa ngõ truy cập toàn bộ hệ thống; có 4 nhánh thông báo lỗi bảo mật rẽ nhánh.
NFR-01	Hiệu năng	4	3	**12**	Cao	Tài khoản glitch bị trễ rõ rệt; trang tải quá lâu khiến người dùng tưởng treo và bỏ giỏ hàng.
FR-02	Giỏ hàng	2	4	**8**	Trung bình	Lỗi hiển thị số lượng badge gây nhầm lẫn số hàng đặt mua.
FR-04	Sắp xếp	2	2	**4**	Thấp	Sắp xếp sai gây khó chịu về trải nghiệm nhưng không làm gián đoạn hay mất tiền.

Bảng 7: Bảng phân tích và đánh giá mức độ rủi ro

Kết luận: Các chức năng có $R \geq 12$ (**FR-03**, **FR-01**, **NFR-01**) được nhóm ưu tiên phân bổ số lượng ca kiểm thử lớn nhất ở NV3.

3.3 Ma trận truy vết yêu cầu (Traceability Matrix)

Ma trận truy vết được nhóm xây dựng ở NV2 và cập nhật đồng bộ xuyên suốt đến cuối bài nộp:

Mã YC	Tên chức năng	Kỹ thuật áp dụng	Mã Test Case	Lớp Test JUnit 5	Mã Lỗi
FR-01	Đăng nhập	EP, Bảng quyết định (DT)	`TC-LOGIN-EP1..EP8`, `DT1..DT5`	`LoginTest`	—
FR-02	Giỏ hàng	BVA, Use Case (UC2)	`TC-CART-B1..B5`, `UC1..UC5`, `P01`	`CartTest`	**BUG-05**
FR-03	Thanh toán	Use Case (UC1), DT, BVA	`TC-CHK-01..05`, `DT1..DT4`, `TAX-B1..B3`, `06..07`	`CheckoutTest`	**BUG-03, 04**
FR-04	Sắp xếp	Phân lớp tương đương (EP)	`TC-SORT-EP1..EP4`	`SortTest`	**BUG-02, 06**
NFR-01	Hiệu năng	Đo thời gian (5 lần đo)	`TC-NFR-01`	`PerformanceTest`	**BUG-01**

Bảng 8: Ma trận truy vết yêu cầu hoàn chỉnh (Requirements Traceability Matrix)

4 NV3 – Thiết kế Test Case

4.1 Phân lớp tương đương (Equivalence Partitioning - EP)

Kỹ thuật EP được nhóm áp dụng trên 2 trường dữ liệu của FR-01 và chức năng sắp xếp FR-04.

4.1.1 Phân lớp tương đương cho trường Username (FR-01)

Mật khẩu cố định là `secret_sauce`. Miền giá trị của Username được chia thành 5 lớp tương đương:

Lớp	Mô tả lớp	Giá trị đại diện	Kết quả mong đợi	Mã TC
EC1 (Hợp lệ)	Tài khoản tồn tại, đang hoạt động	`standard_user`	Chuyển đến URL `inventory.html`	`TC-LOGIN-EP1`
EC2 (Không hợp lệ)	Tài khoản tồn tại nhưng bị khóa	`locked_out_user`	Epic sadface: Sorry, this user has been locked out.	`TC-LOGIN-EP2`
EC3 (Không hợp lệ)	Chuỗi rỗng	`""`	Epic sadface: Username is required	`TC-LOGIN-EP3`
EC4 (Không hợp lệ)	Tài khoản không tồn tại	`abc_user`	Epic sadface: Username and password do not match...	`TC-LOGIN-EP4`
EC5 (Không hợp lệ)	Đúng tên nhưng sai hoa/thường	`Standard_User`	Epic sadface: Username and password do not match...	`TC-LOGIN-EP5`

Bảng 9: Bảng phân lớp tương đương cho trường Username

Nhận xét: Lớp **EC5** đóng vai trò quan trọng vì đặc tả quy định username phân biệt chữ hoa/thường. Nếu bỏ qua lớp này, nhóm không thể phát hiện lỗi nếu hệ thống chấp nhận sai tài khoản viết hoa.

4.1.2 Phân lớp tương đương tự mở rộng cho trường Password (FR-01)

Tên đăng nhập cố định là `standard_user`. Miền giá trị Password được phân thành 4 lớp:

Lớp	Mô tả lớp	Giá trị đại diện	Kết quả mong đợi	Mã TC
EC6 (Hợp lệ)	Mật khẩu chính xác	`secret_sauce`	Chuyển đến URL `inventory.html`	*(Trùng EP1)*
EC7 (Không hợp lệ)	Bỏ trống mật khẩu	`""`	Epic sadface: Password is required	`TC-LOGIN-EP6`
EC8 (Không hợp lệ)	Mật khẩu sai	`wrong_sauce`	Epic sadface: Username and password do not match...	`TC-LOGIN-EP7`
EC9 (Không hợp lệ)	Đúng mật khẩu nhưng sai hoa/thường	`Secret_sauce`	Epic sadface: Username and password do not match...	`TC-LOGIN-EP8`

Bảng 10: Bảng phân lớp tương đương mở rộng cho trường Password

4.1.3 Phân lớp tương đương cho chức năng Sắp xếp (FR-04)

Miền giá trị tùy chọn sắp xếp bao gồm 4 tùy chọn hợp lệ được phân thành 4 ca kiểm thử:

Lớp	Tùy chọn sắp xếp	Giá trị value	Tiêu chí kiểm tra kết quả	Mã TC
EC10	Name (A to Z)	`az`	Tên sản phẩm tăng dần theo thứ tự bảng chữ cái	`TC-SORT-EP1`
EC11	Name (Z to A)	`za`	Tên sản phẩm giảm dần theo thứ tự bảng chữ cái	`TC-SORT-EP2`
EC12	Price (low to high)	`lohi`	Mức giá sản phẩm tăng dần ($7.99 → $49.99)	`TC-SORT-EP3`
EC13	Price (high to low)	`hilo`	Mức giá sản phẩm giảm dần ($49.99 → $7.99)	`TC-SORT-EP4`

Bảng 11: Bảng phân lớp tương đương cho chức năng Sắp xếp FR-04

4.2 Phân tích giá trị biên (Boundary Value Analysis - BVA)

Đề bài yêu cầu áp dụng BVA cho ít nhất 2 miền giá trị với tối thiểu 8 ca kiểm thử.

4.2.1 Miền 1: Số lượng sản phẩm trong giỏ hàng (FR-02)

Cửa hàng SauceDemo có tổng cộng 6 sản phẩm. Do đó, số lượng sản phẩm có thể thêm vào giỏ hàng nằm trong miền hợp lệ $[0, 6]$:

- **Biên dưới:** 0 (giỏ rỗng) và 1 (ngay trên biên dưới: $\min + 1$).
- **Biên trên:** 6 (giỏ đầy) và 5 (ngay dưới biên trên: $\max - 1$).
- **Biên động:** $1 \to 0$ (thêm 1 sản phẩm rồi xóa để kiểm tra việc ẩn badge).

Mã TC	Giá trị biên	Cách tạo dữ liệu thao tác	Kết quả mong đợi
TC-CART-B1	0 (min)	Đăng nhập, không thêm sản phẩm nào	Biểu tượng giỏ hàng rỗng, không hiển thị badge
TC-CART-B2	1 (min+1)	Nhấn Add to cart sản phẩm Backpack	Nút chuyển thành Remove, badge hiển thị số 1
TC-CART-B3	5 (max-1)	Thêm cả 6 sản phẩm rồi nhấn Remove 1	Badge giảm từ 6 về 5, nút đổi lại Add to cart
TC-CART-B4	6 (max)	Nhấn Add to cart cho toàn bộ 6 sản phẩm	Badge hiển thị số 6, cả 6 nút đều là Remove
TC-CART-B5	$1 \to 0$	Thêm 1 sản phẩm rồi nhấn Remove	Badge biến mất hoàn toàn khỏi giao diện

Bảng 12: Bảng phân tích giá trị biên số lượng sản phẩm trong giỏ hàng

Giải thích giá trị ngoài biên (theo yêu cầu đề bài): Các giá trị ngoài biên gồm -1 (dưới biên dưới) và 7 (vượt quá biên trên) **không thể tạo được qua giao diện web**. Lý do là hệ thống không có ô nhập số lượng bằng tay; mỗi sản phẩm chỉ có duy nhất một nút bấm Add to cart/Remove, và toàn bộ cửa hàng chỉ có tối đa 6 sản phẩm độc lập.

4.2.2 Miền 2: Làm tròn thuế 8% tại trang Thanh toán (FR-03)

Theo đặc tả của đề bài, tiền thuế bằng 8% giá trị đơn hàng làm tròn 2 chữ số thập phân. Nhóm thiết kế kiểm thử giá trị biên cho quy tắc làm tròn (làm tròn xuống khi phần thập phân thứ $3 < 5$, làm tròn lên khi ≥ 5, và đúng điểm biên 0.005):

Mã TC	Tổ hợp sản phẩm chọn mua	Item Total	Tính thuế (8%)	Thuế tròn	Ý nghĩa giá trị biên
TC-TAX-B1	Sauce Labs Onesie ($7.99)	$7.99	$7.99 \times 0.08 = 0.6392$	**$0.64**	Phần dư $0.0092 \geq 0.005 \to$ Làm tròn lên
TC-TAX-B2	Bike Light ($9.99) + Bolt T-Shirt ($15.99)	$25.98	$25.98 \times 0.08 = 2.0784$	**$2.08**	Phần dư $0.0084 \geq 0.005 \to$ Làm tròn lên
TC-TAX-B3	Sauce Labs Backpack ($29.99)	$29.99	$29.99 \times 0.08 = 2.3992$	**$2.40**	Giá trị biên điển hình (trùng ví dụ mẫu đề bài)

Bảng 13: Bảng phân tích giá trị biên cho quy tắc làm tròn thuế 8%

4.3 Bảng quyết định (Decision Table - DT)

Đề bài quy định xây dựng tối thiểu 2 bảng quyết định (≥ 8 ca kiểm thử).

4.3.1 Bảng quyết định 1: Logic xác thực Đăng nhập (FR-01)

Bảng quyết định phân tích tổ hợp 4 điều kiện đầu vào của quá trình xác thực:

Điều kiện / Hành động	**R1**	**R2**	**R3**	**R4**	**R5**
C1: Username để trống	**Y**	N	N	N	N
C2: Password để trống	–	**Y**	N	N	N
C3: Cặp username/password đúng	–	–	**N**	**Y**	**Y**
C4: Tài khoản bị khóa	–	–	–	**Y**	**N**
A1: Lỗi: *Username is required*	✓				
A2: Lỗi: *Password is required*		✓			
A3: Lỗi: *Username and password do not match...*			✓		
A4: Lỗi: *Sorry, this user has been locked out.*				✓	
A5: Chuyển đến trang `inventory.html`					✓

Bảng 14: Bảng quyết định logic xác thực người dùng (FR-01)

Mỗi quy tắc từ R1 đến R5 được ánh xạ thành một ca kiểm thử cụ thể:

- **R1** → `TC-LOGIN-DT1:` `user=""`, `pass=""` → Nhận lỗi A1 (*Username is required*).
- **R2** → `TC-LOGIN-DT2:` `user="standard_user"`, `pass=""` → Nhận lỗi A2 (*Password is required*).
- **R3** → `TC-LOGIN-DT3:` `user="locked_out_user"`, `pass="wrong"` → Nhận lỗi A3 (*Username and password do not match...*).
- **R4** → `TC-LOGIN-DT4:` `user="locked_out_user"`, `pass="secret_sauce"` → Nhận lỗi A4 (*Sorry, this user has been locked out.*).
- **R5** → `TC-LOGIN-DT5:` `user="standard_user"`, `pass="secret_sauce"` → Chuyển đến trang sản phẩm (A5).

4.3.2 Bảng quyết định 2: Xác thực biểu mẫu thông tin Thanh toán (FR-03)

Đặc tả quy định First Name, Last Name, Postal Code đều bắt buộc và kiểm tra theo thứ tự ưu tiên:

Điều kiện / Hành động	**R1**	**R2**	**R3**	**R4**
C1: First Name có dữ liệu	**N**	**Y**	**Y**	**Y**
C2: Last Name có dữ liệu	–	**N**	**Y**	**Y**
C3: Postal Code có dữ liệu	–	–	**N**	**Y**
A1: Lỗi: *Error: First Name is required*	✓			
A2: Lỗi: *Error: Last Name is required*		✓		
A3: Lỗi: *Error: Postal Code is required*			✓	
A4: Chuyển sang trang `checkout-step-two.html`				✓

Bảng 15: Bảng quyết định kiểm tra hợp lệ dữ liệu biểu mẫu thanh toán (FR-03)

Ánh xạ thành 4 ca kiểm thử Bảng quyết định cho thanh toán:

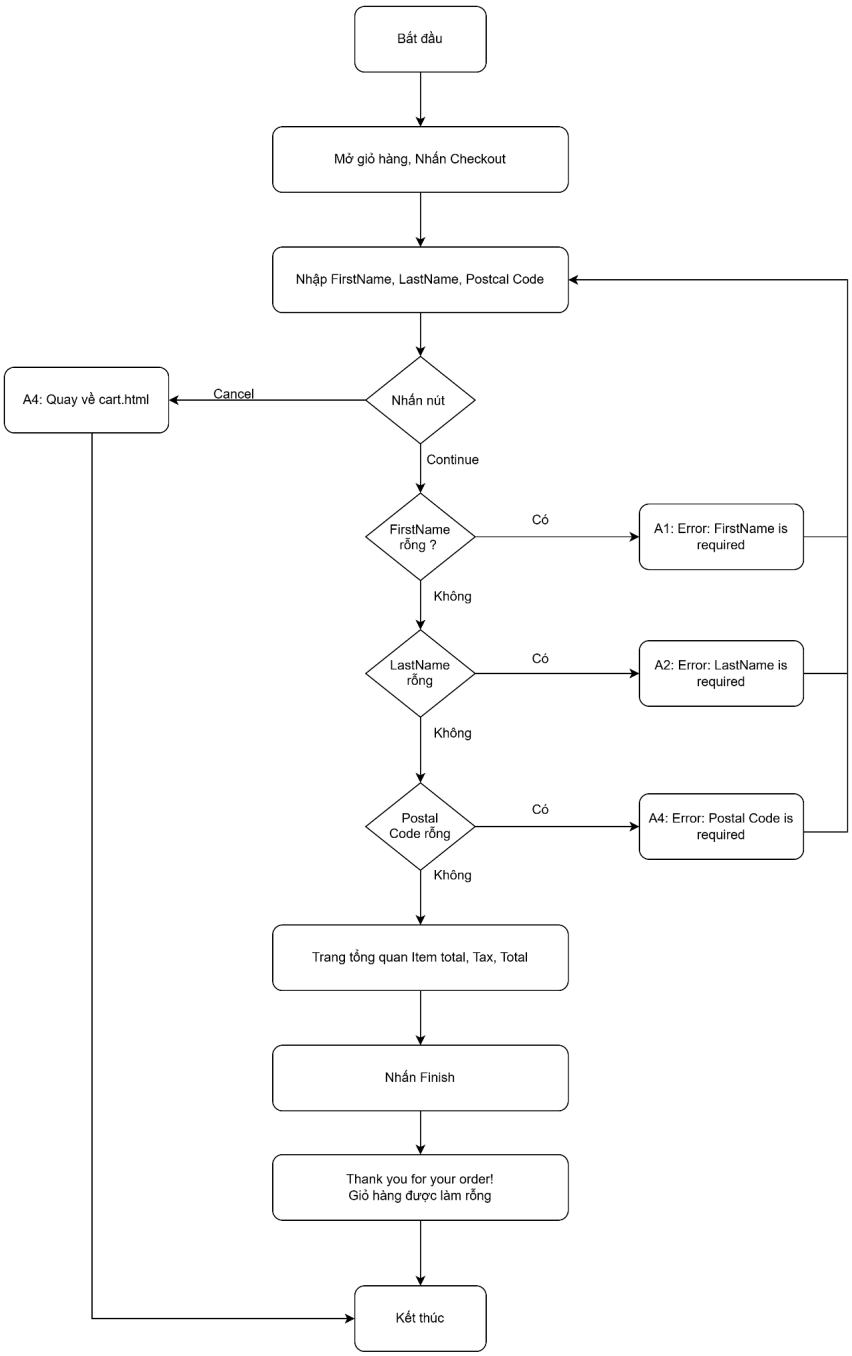
- **R1** → `TC-CHK-DT1:` Bỏ trống First Name → Báo lỗi A1.
- **R2** → `TC-CHK-DT2:` Nhập First Name, bỏ trống Last Name → Báo lỗi A2.
- **R3** → `TC-CHK-DT3:` Nhập First, Last Name, bỏ trống Postal Code → Báo lỗi A3.
- **R4** → `TC-CHK-DT4:` Nhập đủ cả 3 trường → Chuyển tiếp thành công (A4).

4.4 Kiểm thử ca sử dụng (Use Case Testing)

Đề bài yêu cầu áp dụng kỹ thuật Use Case cho 2 ca sử dụng: mỗi ca sử dụng có sơ đồ hoạt động (Activity Diagram), luồng chính và tối thiểu 2 luồng thay thế/ngoại lệ.

4.4.1 Ca sử dụng 1 (UC1): Quy trình thanh toán đơn hàng (FR-03)

Tiền điều kiện: Đã đăng nhập bằng `standard_user`, giỏ hàng có ít nhất một sản phẩm.



Hình 2: Sơ đồ hoạt động (Activity Diagram) của Ca sử dụng Thanh toán đơn hàng (FR-03)

Từ sơ đồ hoạt động, nhóm xây dựng bảng kịch bản kiểm thử bao phủ toàn diện luồng chính và 4 luồng ngoại lệ:

Mã TC	Luồng trên sơ đồ	Dữ liệu đầu vào (First / Last / Postal)	Kết quả mong đợi
`TC-CHK-01`	Luồng chính (Happy Path)	`Duc / Tran / 700000`	Thank you for your order! (Giỏ hàng được làm rỗng)
`TC-CHK-02`	Ngoại lệ A1 (Thiếu First Name)	`"" / Tran / 700000`	Error: First Name is required
`TC-CHK-03`	Ngoại lệ A2 (Thiếu Last Name)	`Duc / "" / 700000`	Error: Last Name is required
`TC-CHK-04`	Ngoại lệ A3 (Thiếu Postal Code)	`Duc / Tran / ""`	Error: Postal Code is required
`TC-CHK-05`	Ngoại lệ A4 (Hủy Cancel)	(Không nhập) / Nhấn nút Cancel	Chuyển về URL `cart.html`, sản phẩm vẫn trong giỏ hàng

Bảng 16: Bảng kịch bản kiểm thử rút ra từ Sơ đồ hoạt động Thanh toán đơn hàng FR-03

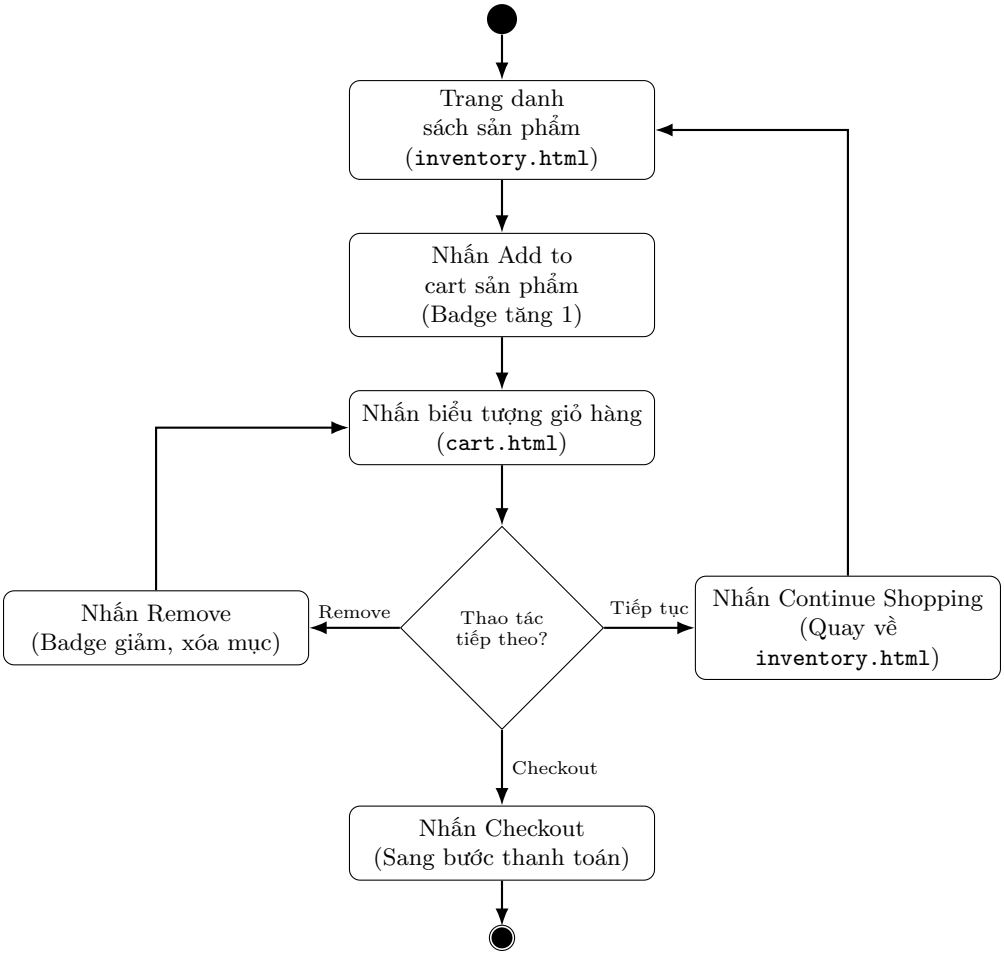
Đặc tả chi tiết từng bước của ca kiểm thử mẫu `TC-CHK-01`:

Bước	Thao tác thực hiện	Kết quả mong đợi chi tiết
1	Thêm sản phẩm Sauce Labs Backpack vào giỏ hàng	Badge giỏ hàng hiển thị số 1; nút chuyển thành Remove
2	Nhấn vào biểu tượng giỏ hàng	Chuyển hướng sang `cart.html`; hiển thị 1 sản phẩm với giá $29.99
3	Nhấn nút Checkout	Chuyển sang URL `checkout-step-one.html`
4	Nhập First Name: `Duc`, Last Name: `Tran`, Postal Code: `700000`; nhấn nút Continue	Chuyển sang URL `checkout-step-two.html`
5	Kiểm tra thông tin tóm tắt thanh toán	Item total: $29.99, Tax: $2.40 ($29.99 \times 8\% \approx 2.40$), Total: $32.39 ($29.99 + 2.40 = 32.39$)
6	Nhấn nút Finish	Chuyển sang URL `checkout-complete.html`; hiển thị thông điệp *"Thank you for your order!"*; giỏ hàng được làm rỗng và badge biến mất

Bảng 17: Đặc tả chi tiết kịch bản kiểm thử `TC-CHK-01`

4.4.2 Ca sử dụng 2 (UC2): Quản lý giỏ hàng (FR-02) [Nhóm tự mở rộng]

Mô tả ca sử dụng: Ca sử dụng thứ hai được nhóm tự xây dựng theo hướng dẫn tại Mục 5.4 của đề bài, bao gồm các luồng: thêm sản phẩm từ trang chính, xem giỏ hàng, xóa sản phẩm trong trang giỏ hàng, và tiếp tục mua sắm (*Continue Shopping*).



Hình 3: Sơ đồ hoạt động (Activity Diagram) của Ca sử dụng 2: Quản lý giỏ hàng

Bảng kịch bản kiểm thử rút ra từ sơ đồ ca sử dụng Quản lý giỏ hàng:

Mã TC	Luồng trên sơ đồ	Thao tác người dùng	Kết quả mong đợi
TC-CART-UC1	Luồng chính (Thêm & Xem)	Nhấn Add to cart tại inventory.html, bấm biểu tượng giỏ	URL là cart.html, sản phẩm hiển thị đúng tên, giá, nút Remove
TC-CART-UC2	Luồng xóa trong giỏ hàng	Đang ở cart.html, nhấn Remove sản phẩm	Sản phẩm bị xóa khỏi danh sách, badge giỏ hàng giảm hoặc biến mất
TC-CART-UC3	Luồng tiếp tục mua sắm	Đang ở cart.html, nhấn nút Continue Shopping	Chuyển về inventory.html, các sản phẩm đã chọn trước đó vẫn còn
TC-CART-UC4	Luồng xóa tại trang chủ	Đã thêm sản phẩm, nhấn nút Remove ngay tại inventory.html	Nút đổi lại thành Add to cart, badge giỏ hàng giảm đi 1
TC-CART-UC5	Luồng chuyển sang Checkout	Đang ở cart.html, nhấn nút Checkout	Chuyển hướng thành công sang trang checkout-step-one.html

Bảng 18: Kịch bản kiểm thử rút ra từ Sơ đồ hoạt động Quản lý giỏ hàng UC2

5 NV4 – Thực thi thủ công và Báo cáo lỗi

5.1 Nhật ký thực thi kiểm thử thủ công

Nhóm tiến hành chạy thực tế toàn bộ các ca kiểm thử trên môi trường chuẩn (Google Chrome 155, Windows 11 64-bit). Dưới đây là bảng trích xuất kết quả thực thi kiểm thử:

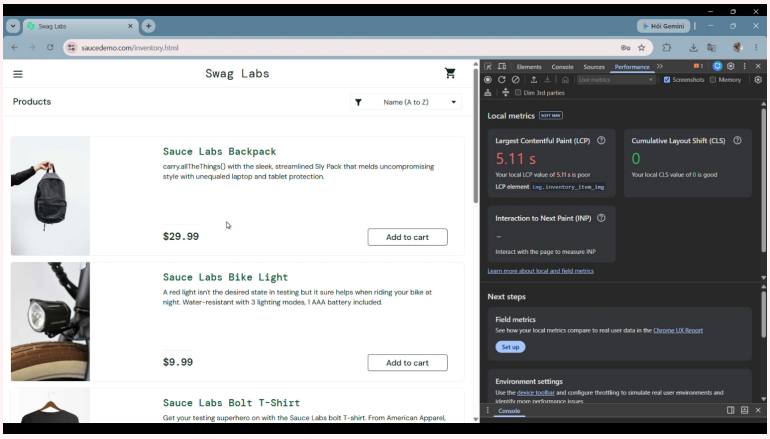
Mã TC	Tên ca kiểm thử	Người chạy	Kết quả thực tế quan sát được	Trạng thái	Mã BUG
TC-LOGIN-EP1..8	Kiểm thử EP Đăng nhập	Khánh Đăng	Chuyển hướng đúng hoặc hiển thị đúng thông báo lỗi	**PASS**	—
TC-SORT-EP1..EP4	Sắp xếp với `standard_user`	Thành Nhân	Danh sách sản phẩm sắp xếp chuẩn xác theo tên và giá	**PASS**	—
TC-SORT-EP2..EP4	Sắp xếp với `problem_user`	Thành Nhân	Dropdown không nhận giá trị, danh sách sản phẩm bị đơ	**FAIL**	**BUG-02**
TC-SORT-EP3..EP4	Sắp xếp giá với `visual_user`	Thành Nhân	Giá bị gán ngẫu nhiên, danh sách hiển thị lộn xộn	**FAIL**	**BUG-06**
TC-CART-B1..B5	Kiểm thử biên BVA giỏ hàng	Chí Tài	Badge và nút Remove hoạt động chính xác (0 đến 6)	**PASS**	—
TC-CART-P01	Xóa hàng với `problem_user`	Chí Tài	Nhấn Remove nhưng nút không đổi và sản phẩm không bị xóa	**FAIL**	**BUG-05**
TC-CHK-01..05	Kiểm thử Use Case thanh toán	Việt Đức	Chuyển hướng đúng, tính thuế chuẩn, hiển thị THANK YOU	**PASS**	—
TC-CHK-06	Checkout với `error_user`	Việt Đức	Ô Last Name không nhận ký tự nhưng form vẫn cho Continue	**FAIL**	**BUG-04**
TC-CHK-07	Checkout với `problem_user`	Việt Đức	Nhập vào ô Last Name thì tự động sửa vào ô First Name	**FAIL**	**BUG-03**
TC-NFR-01	Đo hiệu năng 5 lần đăng nhập	Ngọc Vinh	5.12s, 5.08s, 5.11s, 5.15s, 5.10s (Trung vị: 5.11s)	**FAIL**	**BUG-01**

Bảng 19: Trích xuất kết quả thực thi kiểm thử thủ công trên SauceDemo

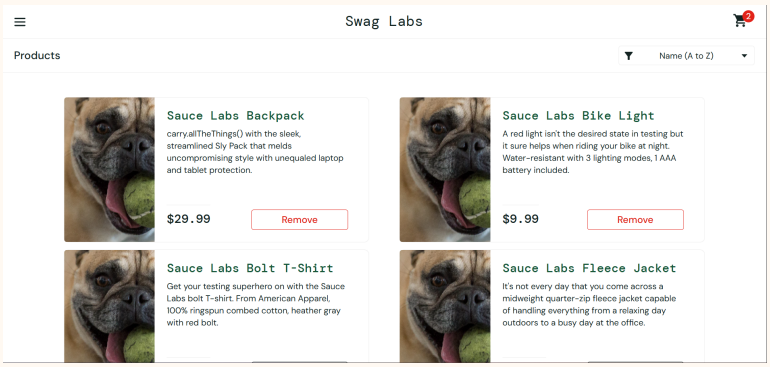
5.2 Báo cáo các lỗi hệ thống phát hiện được (Defect Reports)

Toàn bộ 6 lỗi hệ thống phát hiện được qua kiểm thử khám phá và thực thi đều được lập phiếu báo cáo lỗi chuẩn:

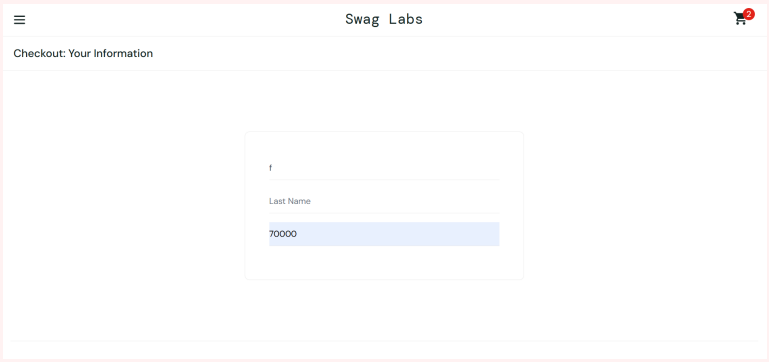
Mã lỗi: BUG-01
Tiêu đề lỗi: Trang sản phẩm mất khoảng 5.11 giây để hiển thị với tài khoản `performance_glitch_user` (vi phạm NFR-01).
Mức nghiêm trọng: Major. **Môi trường:** Google Chrome 155, Win 11, mạng cáp quang.
Các bước tái hiện: 1. Truy cập trang chủ SauceDemo. 2. Nhập `performance_glitch_user` / `secret_sauce`. 3. Bấm Login và đo thời gian đến khi trang tải xong.
Kết quả mong đợi: Trang sản phẩm hiển thị trong thời gian $\leq$ 3.0 giây.
Kết quả thực tế: 5 lần đo: 5.12s, 5.08s, 5.11s, 5.15s, 5.10s (Trung vị: 5.11s). Đo kiểm tra qua công cụ Chrome DevTools Performance ghi nhận chỉ số LCP đạt đúng 5.11 s (mức cảnh báo poor, vượt xa ngưỡng chấp nhận 3.0 s).
Bằng chứng đính kèm: Video ghi hình `evidence/NV4_BUG-01.mp4` và ảnh chụp màn hình DevTools LCP `evidence/NV4_BUG-01.png`.



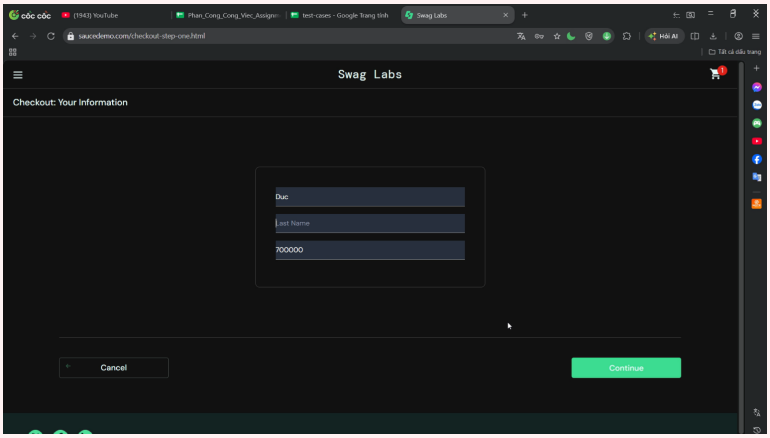
Mã lỗi: BUG-02
Tiêu đề lỗi: Bộ lọc sắp xếp không hoạt động khi chọn các tiêu chí sắp xếp với tài khoản `problem_user`.
Mức nghiêm trọng: Medium. **Môi trường:** Google Chrome 155, Win 11.
Các bước tái hiện: Đăng nhập bằng `problem_user`, nhấp chọn bộ lọc sắp xếp (Z-A hoặc Price low-high/high-low).
Kết quả mong đợi: Dropdown chuyển sang tiêu chí được chọn và danh sách sản phẩm sắp xếp lại tương ứng.
Kết quả thực tế: Dropdown không nhận giá trị mới, danh sách sản phẩm giữ nguyên thứ tự mặc định (Name A to Z).
Bằng chứng đính kèm: Ảnh chụp màn hình `evidence/NV4_BUG-02.png`.



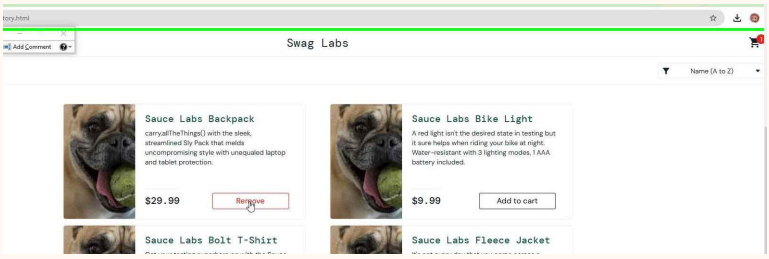
Mã lỗi: BUG-03
Tiêu đề lỗi: Nhập ký tự vào ô Last Name thì hệ thống tự động ghi đè sang ô First Name khi dùng `problem_user`.
Mức nghiêm trọng: Major. **Môi trường:** Google Chrome 155, Win 11.
Các bước tái hiện: Đăng nhập bằng `problem_user`, vào Checkout Step One, nhập First Name: 'An', sau đó nhập Last Name: 'Nguyen'.
Kết quả mong đợi: Ô First Name giữ nguyên 'An', ô Last Name hiển thị 'Nguyen'.
Kết quả thực tế: Ô First Name bị tự động ghi đè thành 'Nguyen', ô Last Name không nhận giá trị độc lập.
Bằng chứng đính kèm: Ảnh chụp màn hình `evidence/NV4_BUG-03.png`.



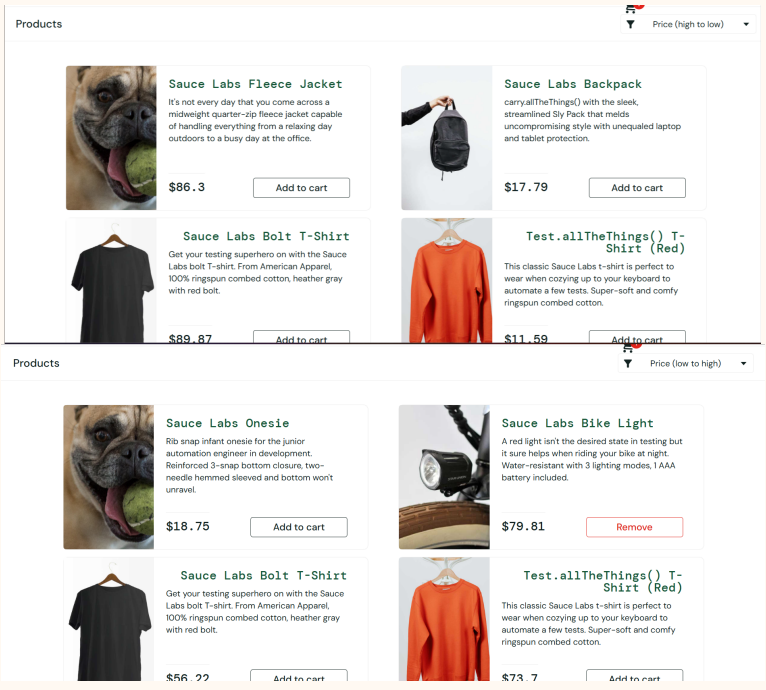
Mã lỗi: BUG-04
Tiêu đề lỗi: Ô Last Name không nhận ký tự nhưng form vẫn cho phép Continue sang Overview khi dùng `error_user`.
Mức nghiêm trọng: Major. **Môi trường:** Google Chrome 155, Win 11.
Các bước tái hiện: Đăng nhập `error_user`, vào Checkout Step One, nhập First Name: 'Duc', Postal Code: '700000', để trống Last Name và nhấn Continue.
Kết quả mong đợi: Ô Last Name nhận ký tự bình thường; nếu thiếu Last Name phải chặn lại và hiển thị lỗi.
Kết quả thực tế: Ô Last Name không gõ được chữ (Console báo lỗi TypeError); khi nhấn Continue hệ thống bypass validation và tự chuyển sang `checkout-step-two.html`.
Bằng chứng đính kèm: Video ghi hình `evidence/NV4_BUG-04.mp4` và ảnh chụp màn hình `evidence/NV4_BUG-04.png`.



Mã lỗi: BUG-05
Tiêu đề lỗi: Không thể xóa sản phẩm khỏi giỏ hàng bằng nút Remove khi dùng tài khoản `problem_user`.
Mức nghiêm trọng: Major. **Môi trường:** Google Chrome 155, Win 11.
Các bước tái hiện: Đăng nhập `problem_user`, thêm sản phẩm vào giỏ, vào `cart.html` và nhấn nút Remove.
Kết quả mong đợi: Sản phẩm bị xóa khỏi giỏ, nút đổi lại thành 'Add to cart', badge giỏ hàng giảm đi 1.
Kết quả thực tế: Nút không chuyển lại thành Add to cart, sản phẩm vẫn còn trong giỏ và badge không giảm.
Bằng chứng đính kèm: Ảnh chụp màn hình `evidence/NV4_BUG-05.png`.



Mã lỗi: BUG-06
Tiêu đề lỗi: Giá sản phẩm bị hiển thị ngẫu nhiên và danh sách giá bị lộn xộn khi dùng `visual_user`.
Mức nghiêm trọng: Medium. **Môi trường:** Google Chrome 155, Win 11.
Các bước tái hiện: Đăng nhập `visual_user`, quan sát giá niêm yết trên `inventory.html` và chọn bộ lọc theo giá.
Kết quả mong đợi: Giá sản phẩm hiển thị đúng giá niêm yết gốc và được sắp xếp tuần tự.
Kết quả thực tế: Giá sản phẩm bị gán số ngẫu nhiên; khi lọc theo giá thì thứ tự hiển thị bị lộn xộn.
Bằng chứng đính kèm: Ảnh chụp màn hình `evidence/NV4_BUG-06.png`.



6 NV5 – Tự động hóa bằng JUnit 5

6.1 Kiến trúc Page Object Model (POM)

Nhóm triển khai mô hình thiết kế chuẩn công nghiệp Page Object Model:

- Tách biệt hoàn toàn giữa **Phần tử giao diện & Thao tác người dùng** (nằm trong thư mục `pages/`) và **Mã kiểm thử & Khẳng định kết quả** (nằm trong thư mục `tests/`).
- Tuyệt đối tuân thủ quy tắc của đề bài: **Không sử dụng** `Thread.sleep()`, 100% các thao tác đồng bộ đều dùng cơ chế chờ tường minh **Explicit Wait** (`WebDriverWait` **kết hợp** `ExpectedConditions`).

6.2 Trích đoạn mã nguồn các lớp kiểm thử tiêu biểu

6.2.1 Lớp LoginTest.java (Sử dụng CsvFileSource và CsvSource)

Listing 1: Trích đoạn mã nguồn lớp LoginTest.java

```
@Tag("web")
@DisplayName("FR-01: Authentication (Login)")
class LoginTest extends WebTestBase {

    private LoginPage loginPage;

    @BeforeEach
    void openLoginPage() {
        loginPage = new LoginPage(driver, baseUrl).open();
    }

    @ParameterizedTest(name = "{0}: user=''{1}'', accepted={3}")
    @CsvFileSource(resources = "/login-data.csv", numLinesToSkip = 1)
    void loginEquivalencePartitioning(
            String testCaseId,
            String username,
            String password,
            boolean expectedAccepted,
            String expectedEvidence) {

        LoginResult actual = loginPage.login(username, password);

        assertEquals(expectedAccepted, actual.accepted(),
                () -> testCaseId + ": Kết quả accepted không khớp cho user: '" + username + "'");
        assertTrue(
                actual.currentUrl().contains(expectedEvidence)
                        || actual.message().contains(expectedEvidence),
                () -> testCaseId + ": Kỳ vọng chứa '" + expectedEvidence
                        + "', nhưng thực tế URL: '" + actual.currentUrl()
                        + "', thông báo: '" + actual.message() + "'");
    }

    @ParameterizedTest(name = "{0} [{1}]: user=''{2}'', pass=''{3}'' -> error/URL: {5}")
    @CsvSource({
        "TC-LOGIN-DT1, Quy tắc R1, '', '', false, Username is required",
        "TC-LOGIN-DT2, Quy tắc R2, standard_user, '', false, Password is required",
        "TC-LOGIN-DT3, Quy tắc
            R3, locked_out_user, wrong_password, false, Username and password do not match",
        "TC-LOGIN-DT4, Quy tắc
            R4, locked_out_user, secret_sauce, false, Sorry, this user has been locked out",
        "TC-LOGIN-DT5, Quy tắc R5, standard_user, secret_sauce, true, inventory.html"
    })
    void loginDecisionTable(
            String testCaseId,
            String ruleName,
            String username,
```

```
            String password,
            boolean expectedAccepted,
            String expectedEvidence) {

        LoginResult actual = loginPage.login(username, password);

        assertEquals(expectedAccepted, actual.accepted(),
                () -> testCaseId + " (" + ruleName + "): Kết quả accepted không khớp");
        assertTrue(
                actual.currentUrl().contains(expectedEvidence)
                        || actual.message().contains(expectedEvidence),
                () -> testCaseId + " (" + ruleName + "): Kỳ vọng chứa '" + expectedEvidence
                        + "', nhưng thực tế URL: '" + actual.currentUrl()
                        + "', thông báo: '" + actual.message() + "'");
    }
}
```

6.2.2 Lớp CheckoutTest.java (Kiểm thử kịch bản Use Case)

Listing 2: Trích đoạn mã nguồn lớp CheckoutTest.java

```
public class CheckoutTest extends WebTestBase {
    private CheckoutPage checkoutPage;
    private CartPage cartPage;

    @BeforeEach
    void prepareCheckout() {
        LoginPage loginPage = new LoginPage(driver, baseUrl);
        LoginPage.LoginResult loginResult = loginPage.open().login("standard_user", "secret_sauce");
        assertTrue(loginResult.accepted(), "Đăng nhập thất bại:" + loginResult.message());

        InventoryPage inventoryPage = new InventoryPage(driver);
        inventoryPage.open().addToCart("sauce-labs-backpack");
        inventoryPage.openCart();
        cartPage = new CartPage(driver);
        cartPage.open();
        cartPage.clickCheckOut();
        checkoutPage = new CheckoutPage(driver);
    }

    @Test
    @DisplayName("TC-CHK-01 - Checkout thành công")
    void testCheckoutSucces() {
        checkoutPage.fillInformation("Nguyen", "Van A", "700000");
        checkoutPage.continueToOverviewAndReadTotal();
        assertEquals("Item total: $29.99", checkoutPage.getItemTotal());
        assertEquals("Tax: $2.40", checkoutPage.getTax());
        assertEquals("Total: $32.39", checkoutPage.getTotal());
        assertEquals("Thank you for your order!", checkoutPage.finishOrder());
    }

    @Test
    @DisplayName("TC-CHK-02 - Bỏ trống trường FirstName")
    void testMissingFirstName() {
        checkoutPage.fillInformation("", "Van A", "700000");
        assertEquals("Error: First Name is required", checkoutPage.continueExpectingError());
    }

    @Test
    @DisplayName("TC-CHK-03 - Bỏ trống trường LastName")
    void testMissingLastName() {
```

```
        checkoutPage.fillInformation("Nguyen", "", "700000");
        assertEquals("Error: Last Name is required", checkoutPage.continueExpectingError());
    }

    @Test
    @DisplayName("TC-CHK-04 - Bỏ trống trường Postal")
    void testMissingPostal() {
        checkoutPage.fillInformation("Nguyen", "Van A", "");
        assertEquals("Error: Postal Code is required", checkoutPage.continueExpectingError());
    }

    @Test
    @DisplayName("TC-CHK-05 - Nhấn Cancel Checkout ")
    void testCancelCheckout() {
        checkoutPage.clickCancel();
        cartPage.open();
        assertTrue(driver.getCurrentUrl().contains("cart.html"), "Không quay lại giỏ hàng");
        assertTrue(cartPage.isProductInCart("Sauce Labs Backpack"), "Sản phẩm đã mất");
        assertEquals(1, cartPage.countProduct(), "Số lượng sản phẩm không đúng ");
    }
}
```

6.2.3 Lớp CartTest.java (Kiểm thử giá trị biên BVA giỏ hàng - FR-02)

Listing 3: Trích đoạn mã nguồn lớp CartTest.java

```
@TestMethodOrder(MethodOrderer.DisplayName.class)
@DisplayName("FR-02: Kiểm thử giá trị biên của giỏ hàng")
public class CartTest extends WebTestBase {

    private InventoryPage inventory;

    private static final String[] PRODUCTS = {
        "sauce-labs-backpack",
        "sauce-labs-bike-light",
        "sauce-labs-bolt-t-shirt",
        "sauce-labs-fleece-jacket",
        "sauce-labs-onesie",
        "test.allthethings()-t-shirt-(red)"
    };

    @BeforeEach
    void setUpCartTest() {
        new LoginPage(driver, baseUrl)
                .open()
                .login("standard_user", "secret_sauce");

        inventory = new InventoryPage(driver).open();
    }

    @Test
    @DisplayName("TC-CART-B1: Giỏ rỗng không hiển thị badge")
    void emptyCartHasNoBadge() {
        assertEquals(0, inventory.cartBadgeCount(), "Badge phải không xuất hiện khi giỏ hàng rỗng");
    }

    @Test
    @DisplayName("TC-CART-B2: Thêm 1 sản phẩm, badge bằng 1")
    void addOneProduct() {
        inventory.addToCart(PRODUCTS[0]);
        assertEquals(1, inventory.cartBadgeCount(), "Badge phải hiển thị số 1");
```

```
    }

    @Test
    @DisplayName("TC-CART-B3: Giỏ có 5 sản phẩm, badge bằng 5")
    void cartHasFiveProducts() {
        for (String product : PRODUCTS) {
            inventory.addToCart(product);
        }
        inventory.removeFromCart(PRODUCTS[5]);
        inventory.waitForCartBadge("5");
        assertEquals(5, inventory.cartBadgeCount(), "Badge phải hiển thị số 5");
    }

    @Test
    @DisplayName("TC-CART-B4: Thêm đủ 6 sản phẩm, badge bằng 6")
    void cartHasSixProducts() {
        for (String product : PRODUCTS) {
            inventory.addToCart(product);
        }
        assertEquals(6, inventory.cartBadgeCount(), "Badge phải hiển thị số 6");
    }

    @Test
    @DisplayName("TC-CART-B5: Xóa sản phẩm cuối cùng, badge biến mất")
    void removeLastProduct() {
        inventory.addToCart(PRODUCTS[0]);
        assertEquals(1, inventory.cartBadgeCount(), "Trước khi xóa, badge phải bằng 1");
        inventory.removeFromCart(PRODUCTS[0]);
        inventory.waitForCartBadge("0");
        assertEquals(0, inventory.cartBadgeCount(), "Badge phải biến mất khi giỏ hàng trở về 0");
    }
}
```

6.2.4 Lớp SortTest.java (Kiểm thử sắp xếp và phát hiện lỗi Sort - FR-04)

Listing 4: Trích đoạn mã nguồn lớp SortTest.java

```
@Tag("web")
@DisplayName("FR-04: Sort")
public class SortTest extends WebTestBase {

    private InventoryPage loginInventoryPage(String username) {
        LoginPage loginPage = new LoginPage(driver, baseUrl).open();
        LoginResult result = loginPage.login(username, "secret_sauce");
        assertTrue(result.accepted(), "Đăng nhập thất bại với tài khoản: " + username);
        return new InventoryPage(driver).open();
    }

    @ParameterizedTest(name = "{0}: user={1}, option={2}")
    @DisplayName("FR-04: Sắp xếp sản phẩm theo tên (A-Z, Z-A)")
    @CsvSource({
        "TC-SORT-EP1, standard_user, az, true",
        "TC-SORT-EP2, standard_user, za, false",
        "TC-SORT-EP1, problem_user, az, true",
        "TC-SORT-EP1, visual_user, az, true",
        "TC-SORT-EP2, visual_user, za, false"
    })
    void sortByNamePass(String tcId, String username, String option, boolean ascending) {
        InventoryPage inventoryPage = loginInventoryPage(username);
        List<String> actual = inventoryPage.sortBy(option).names();
        List<String> expected = new ArrayList<>(actual);
```

```
        expected.sort(ascending ? Comparator.naturalOrder() : Comparator.reverseOrder());
        assertEquals(expected, actual, () -> String.format("[%s] Thứ tự tên sản phẩm không đúng
            với user=%s, option=%s", tcId, username, option));
    }

    @ParameterizedTest(name = "{0}: user={1}, option={2}")
    @DisplayName("FR-04: Sắp xếp sản phẩm theo giá (low-high, high-low)")
    @CsvSource({
        "TC-SORT-EP3, standard_user, lohi, true",
        "TC-SORT-EP4, standard_user, hilo, false"
    })
    void sortByPricePass(String tcId, String username, String option, boolean ascending) {
        InventoryPage inventoryPage = loginInventoryPage(username);
        List<Double> actual = inventoryPage.sortBy(option).prices();
        List<Double> expected = new ArrayList<>(actual);
        expected.sort(ascending ? Comparator.naturalOrder() : Comparator.reverseOrder());
        assertEquals(expected, actual, () -> String.format("[%s] Thứ tự giá sản phẩm không đúng
            với user=%s, option=%s", tcId, username, option));
    }

    @Tag("known-bug")
    @ParameterizedTest(name = "{0}: user={1}, option={2}")
    @DisplayName("FR-04: Lỗi sắp xếp theo tên Z-A với problem_user")
    @CsvSource({
        "TC-SORT-EP2[BUG-02], problem_user, za, false"
    })
    void sortByNameKnownBugs(String tcId, String username, String option, boolean ascending) {
        InventoryPage inventoryPage = loginInventoryPage(username);
        List<String> actual = inventoryPage.sortBy(option).names();
        List<String> expected = new ArrayList<>(actual);
        expected.sort(ascending ? Comparator.naturalOrder() : Comparator.reverseOrder());
        assertEquals(expected, actual, () -> String.format("[%s] Hệ thống không sắp xếp tên
            theo %s cho %s", tcId, option, username));
    }

    @Tag("known-bug")
    @ParameterizedTest(name = "{0}: user={1}, option={2}")
    @DisplayName("FR-04: Lỗi sắp xếp theo giá với problem_user và visual_user")
    @CsvSource({
        "TC-SORT-EP3[BUG-02], problem_user, lohi, true",
        "TC-SORT-EP4[BUG-02], problem_user, hilo, false",
        "TC-SORT-EP3[BUG-06], visual_user, lohi, true",
        "TC-SORT-EP4[BUG-06], visual_user, hilo, false"
    })
    void sortByPriceKnownBugs(String tcId, String username, String option, boolean ascending) {
        InventoryPage inventoryPage = loginInventoryPage(username);
        List<Double> actual = inventoryPage.sortBy(option).prices();
        List<Double> expected = new ArrayList<>(actual);
        expected.sort(ascending ? Comparator.naturalOrder() : Comparator.reverseOrder());
        assertEquals(expected, actual, () -> String.format("[%s] Hệ thống không sắp xếp giá
            theo %s cho %s", tcId, option, username));
    }
}
```

6.2.5 Lớp PerformanceTest.java (Đo hiệu năng NFR-01 & Đánh dấu lỗi đã biết)

Listing 5: Trích đoạn mã nguồn lớp PerformanceTest.java

```
class PerformanceTest extends WebTestBase {

    @Test
    @Tag("known-bug")
```

```
    @DisplayName("TC-NFR-01: Login performance")
    void inventoryLoadsWithin3Seconds() {
        LoginPage loginPage = new LoginPage(driver, baseUrl).open();
        long start = System.nanoTime();
        LoginResult result = loginPage.login("performance_glitch_user", "secret_sauce");
        long elapsedMs = Duration.ofNanos(System.nanoTime() - start).toMillis();

        System.out.println("Thời gian đăng nhập: " + elapsedMs + " ms");
        System.out.println("Kết quả: " + result.message());

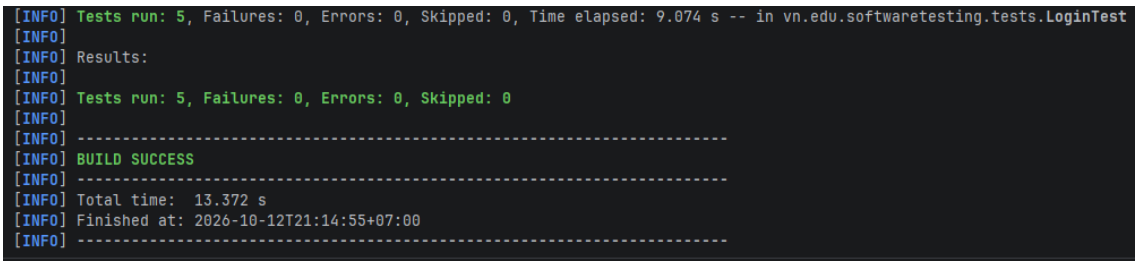
        assertTrue(result.accepted(), "Đăng nhập không thành công");
        assertTrue(elapsedMs <= 3000, "Vượt ngưỡng 3000 ms: " + elapsedMs + " ms");
    }
}
```

6.3 Kết quả thực thi tự động (Maven Surefire Report)

Nhóm thực thi lệnh kiểm tra tự động hóa qua Maven:

```
mvn test -DexcludedGroups=known-bug
[INFO] Running vn.edu.softwaretesting.tests.LoginTest
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 24.57 s
[INFO] Running vn.edu.softwaretesting.tests.CheckoutTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 10.86 s
[INFO] Running vn.edu.softwaretesting.tests.CartTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 11.07 s
[INFO] Running vn.edu.softwaretesting.tests.SortTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 28.60 s
[INFO] ------------------------------------
[INFO] Results:
[INFO] Tests run: 30, Failures: 0, Errors: 0, Skipped: 0
[INFO] ------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------
```

Khi chạy kiểm thử nhóm lỗi đã biết (`mvn test -Dgroups=known-bug`), các ca kiểm thử tái hiện lỗi (`PerformanceTest`, `CheckoutProblemUserTest`, `CheckoutErrorUserTest`, `SortTest`) ghi nhận FAIL chính xác nhằm minh chứng sự tồn tại của 6 BUG theo đúng yêu cầu đề bài.



Hình 4: Ảnh chụp màn hình kết quả thực thi tự động hóa với Maven

6.4 Cấu hình mở rộng đạt điểm Bonus (Mục 7.7 đề bài)

Theo quy định tại Mục 7.7 của đề bài, nhóm đã nghiên cứu và triển khai thành công hai tính năng nâng cao nhằm đạt điểm cộng tối đa:

6.4.1 Cơ chế tự động chụp ảnh màn hình khi FAIL bằng JUnit 5 TestWatcher

Nhóm triển khai lớp `ScreenshotWatcher` hiện thực đồng thời hai giao diện cốt lõi của JUnit 5 Extension: `TestWatcher` và `AfterTestExecutionCallback`. Khi một ca kiểm thử bị thất bại (tiêu biểu khi chạy nhóm kiểm thử mang nhãn `known-bug`), callback sẽ can thiệp ngay thời điểm phát sinh lỗi trước khi trình duyệt bị đóng

bởi `@AfterEach`, sử dụng `TakesScreenshot` của Selenium để chụp lại nguyên trạng màn hình trình duyệt và tự động lưu vào thư mục `target/screenshots/` theo định dạng `{ClassName}_{MethodName}_{Timestamp}.png`. Lớp cơ sở `WebTestBase` được gắn annotation `@ExtendWith(ScreenshotWatcher.class)`, giúp toàn bộ các lớp kiểm thử tự động thừa hưởng tính năng này một cách trong suốt mà không cần sửa đổi mã nguồn kiểm thử.

6.4.2 Tích hợp quy trình kiểm thử tự động liên tục (CI) với GitHub Actions

Nhóm xây dựng quy trình CI hoàn chỉnh thông qua tệp cấu hình `.github/workflows/test.yml`:

- **Môi trường thực thi:** Máy ảo Ubuntu (`ubuntu-latest`) tích hợp Google Chrome Headless và bộ phát triển OpenJDK 17.
- **Kích hoạt tự động:** Workflow tự động kích hoạt mỗi khi có thao tác `push` hoặc `pull_request` vào nhánh chính `main`.
- **Lưu trữ kết quả kiểm thử (Artifacts):** Sử dụng `actions/upload-artifact@v4` để tự động đóng gói toàn bộ thư mục báo cáo `target/surefire-reports/` thành artifact tải về trên giao diện GitHub Actions với thời hạn lưu trữ 14 ngày. Ngoài ra, nếu có kiểm thử bị FAIL, thư mục ảnh chụp `target/screenshots/` từ `TestWatcher` cũng được tự động lưu trữ làm bằng chứng phân tích lỗi.

7 Kết luận và Tự đánh giá trước khi nộp bài

Đối chiếu với danh sách kiểm tra (Checklist Mục 9 trong đề bài của giảng viên Từ Lãng Phiêu):

- ✓ Lệnh `mvn clean test` thực thi trơn tru, độc lập trên máy sạch; chỉ có duy nhất các kiểm thử mang nhãn `@Tag("known-bug")` được phép FAIL.
- ✓ 100% các yêu cầu chức năng (FR-01 đến FR-04) và phi chức năng (NFR-01) đều có tối thiểu một ca kiểm thử bao phủ.
- ✓ Mỗi ca kiểm thử bị FAIL trong thực tế đều có mã định danh lỗi **BUG-xx** tương ứng trong báo cáo lỗi.
- ✓ Toàn bộ kết quả mong đợi đều được tính toán khoa học từ tài liệu đặc tả, không chép lại từ màn hình giao diện.
- ✓ Mã định danh ca kiểm thử đồng nhất 100% giữa tệp Excel (`test-cases.xlsx`), báo cáo thuyết minh PDF và thuộc tính `@DisplayName` trong mã nguồn tự động hóa.
- ✓ Toàn bộ mã nguồn kiểm thử không có bất kỳ dòng lệnh `Thread.sleep()` nào, tuân thủ nghiêm ngặt cơ chế Explicit Wait.
- ✓ [**Bonus đạt chuẩn**] Cấu hình hoàn chỉnh JUnit 5 `TestWatcher` tự động chụp ảnh màn hình khi FAIL và lưu tại `target/screenshots/`.
- ✓ [**Bonus đạt chuẩn**] Thiết lập pipeline GitHub Actions CI tự động hóa chạy kiểm thử và lưu trữ `surefire-reports` làm artifact.